

ADR 002 - Comunicação síncrona via HTTP REST entre ms-checkout e ms-pagamento

Data: 11/08/2026 (Entrega) | **Status:** Aceito

Contexto

Com a separação definida no ADR 0001, o ms-checkout precisa saber, antes de responder ao cliente, se o pagamento foi aprovado ou recusado. Essa informação define o status do pedido (PAGO/FALHA) e as etapas exibidas na confirmação (nota fiscal, e-mail). Como o pagamento neste MVP é simulado internamente (sem integração real com um gateway externo), não existe um tempo de resposta variável de terceiros que justificasse desacoplar a chamada.

O prazo do projeto e o escopo de equipe pequena também pesaram na decisão: operar um Message Broker exige infraestrutura adicional (subir o broker, configurar exchange/queue, implementar publisher e consumer, tratar idempotência) que a equipe não teria tempo de implementar e testar de forma confiável até a entrega, além de tornar a demonstração em vídeo/documento mais difícil de reproduzir por quem for avaliar.

Decisão

Adotar comunicação síncrona via HTTP REST, usando o RestClient do Spring, para a chamada do ms-checkout ao ms-pagamento. Cada PaymentStrategy (CardStrategy, PixStrategy) faz uma requisição POST bloqueante para `http://localhost:8080/api/pagamentos` e aguarda a resposta antes de o CheckoutTemplate continuar o fluxo (`processDebit >>> definir status >>> saveOrderStatus >>> sendReceipt`). Toda a sequência de etapas do pedido é montada dentro do mesmo ciclo de requisição/resposta do checkout.

Consequências

Positivo:

- Simplicidade de implementação e operação: nenhuma infraestrutura adicional além dos dois serviços Spring Boot.
- Consistência imediata: o ms-checkout sempre responde ao cliente já sabendo o status real do pagamento, sem estados intermediários de "pendente".
- Facilita a demonstração e a depuração. O fluxo inteiro (autenticação >>> pagamento >>> persistência >>> recibo) é rastreável em uma única requisição/resposta.

Negativo:

- Acoplamento temporal entre os serviços: se o ms-pagamento estiver indisponível ou lento, o ms-checkout fica bloqueado esperando, sem timeout, retry ou circuit breaker implementados.

- A URL do ms-pagamento está hardcoded (<http://localhost:8080/api/pagamentos>) nas Strategies, em vez de externalizada em configuração (application.properties/variável de ambiente), uma limitação conhecida que dificultaria trocar de ambiente (dev/produção) sem recompilar.
- Escalar o ms-checkout não desacopla sua carga do ms-pagamento, já que cada requisição de checkout ainda depende de uma chamada síncrona a ele.
- Caso o projeto evolua para um gateway de pagamento externo real, o tempo de resposta desse gateway passaria a ser diretamente exposto ao usuário na tela de checkout, um ponto que motivaria revisar esta decisão em favor de mensageria assíncrona.